

Control Tiempos Eclipse

Documento técnico y funcional

App nativa iPhone para fotógrafos de eclipses solares

Cálculo 100 % offline · Avisos de voz en tiempo real

Motor besseliano · SwiftUI + SwiftData · iOS 17

Eclipse total · 12 de agosto de 2026 · España

Franja de totalidad: Lleida – Zaragoza – Teruel

Francisco Cornellana Castells

cornellana@mac.com

1. Introducción y propósito

Control Tiempos Eclipse es una aplicación nativa de iPhone diseñada específicamente para fotógrafos de eclipses solares. Su objetivo principal es liberar al fotógrafo de la necesidad de mirar el teléfono durante el evento: la app calcula las fases del eclipse a partir de las coordenadas del observador y ejecuta un programa de avisos de voz pronunciados en el segundo exacto, sin conexión a internet.

El primer uso real es el eclipse total del 12 de agosto de 2026 visible desde España (franja Lleida–Zaragoza–Teruel). La app está diseñada para funcionar completamente offline: el cálculo de efemérides usa elementos besselianos embebidos en el bundle.

Problema que resuelve

- La totalidad de un eclipse dura entre 2 y 5 minutos — no hay margen para errores.
- El fotógrafo debe operar varias cámaras con ajustes específicos en momentos exactos.
- Mirar el teléfono durante la totalidad supone perder segundos críticos e irre recuperables.
- Las calculadoras online no ofrecen avisos de voz programables ni funcionan sin red.

Solución implementada

- Cálculo offline de contactos C1–C4, máximo y duración de totalidad.
- Magnitud (fracción del diámetro) y cobertura (fracción del área del disco solar).
- Programa de avisos personalizable: texto, fase de referencia y offset en segundos.
- Pronunciación de avisos con AVSpeechSynthesizer en el segundo exacto (error < 0,3 s).
- Pantalla de ejecución con cuenta atrás en tipografía grande — uso sin mirar.
- Modo simulación para ensayar el programa sin eclipse real.
- HelpSheet: glosario rápido accesible desde la tarjeta de eclipse (C1–C4, magnitud, cobertura, tipos).
- Backup/restore de programas entre dispositivos (.cteprog, share/AirDrop).
- Traducción automática de textos de avisos vía API de Claude cuando el idioma difiere.

2. Usuario objetivo

Usuario:	Francisco Cornellana Castells, fotógrafo y programador avanzado
Idioma nativo:	Català / Español (interfaz: es-ES, ca-ES, en-GB)
Evento real:	Eclipse total del 12 de agosto de 2026, España (Lleida–Zaragoza)
Ensayo general:	Primera semana de agosto de 2026 con cámaras reales
Congelación:	9 de agosto de 2026 — sin cambios de código tras esa fecha

3. Flujo funcional — máquina de estados

La app implementa una máquina de estados lineal de cinco pasos. Interrumpir la ejecución desde cualquier punto vuelve siempre al paso 1. La navegación es unidireccional: solo se puede retroceder al paso inmediatamente anterior.

```
enum AppStep { case location, eventDate, verification, program, execution }
```

Paso 1 — Localización

- Dos vías paralelas: GPS puntual (CoreLocation, When In Use) y coordenadas decimales manuales.
- GPS: una sola lectura con `kCLLocationAccuracyHundredMeters`. Pulsar el botón limpia los campos manuales.
- Campos manuales: lat/lon en grados decimales ($\pm DD.DDDDD$), validación en tiempo real.
- Prioridad: las coordenadas manuales siempre tienen prioridad sobre GPS.
- Localizaciones guardadas: guardar con nombre (SwiftData), recuperar, sobrescribir o eliminar.
- Seleccionar una ubicación guardada rellena los campos manuales y muestra su nombre en el paso 3.

Paso 2 — Fecha del evento

- DatePicker gráfico (solo fecha) con fecha por defecto = próximo eclipse visible desde esa posición.
- Botón 'Today' para seleccionar la fecha actual — útil en ensayos.

Paso 3 — Verificación sobre mapa

- Mapa MapKit centrado en la coordenada activa (manual > GPS). No es un selector — es solo visual.
- El título del header muestra el nombre de la localización guardada (si se usó una).
- Eclipse encontrado → tarjeta con C1, C2, MAX, C3, C4, magnitud, duración de totalidad, altitud y azimut solar en el máximo. Horas en zona horaria del dispositivo.
- Sin eclipse → modo SIMULACIÓN con badge visible: el usuario introduce la hora C2 asumida. Fases ficticias: $C1 = C2 - 60 \text{ min}$, $MAX = C2 + 40 \text{ s}$, $C3 = C2 + 80 \text{ s}$, $C4 = C2 + 60 \text{ min}$.

Paso 4 — Programa de eventos

- Lista de eventos ordenada por tiempo absoluto resuelto contra las fases reales o simuladas.
- Cada evento: fase de referencia (C1/C2/MAX/C3/C4), offset en segundos (+/-) y texto del aviso.
- Vista de fila (2 líneas): línea 1 = fase + mensaje + hora resuelta; línea 2 = offset y gap al anterior.
- CRUD completo: añadir, editar, eliminar (swipe), borrar todo (con confirmación).
- Cambiar localización o fecha recalcula automáticamente todos los tiempos del programa.
- Programas con nombre: guardar, cargar, sobrescribir y eliminar programas completos (SwiftData).
- El título de la NavigationBar muestra el nombre del programa activo.
- Badge dorado en los avisos cuyo idioma difiere del idioma de voz (serán traducidos).

Paso 5 — Ejecución

- Fondo: mapa MapKit de la localización a pantalla completa (no interactivo).
- Overlay con material translúcido (`.ultraThinMaterial`): lista de eventos con el próximo centrado.
- Cuenta atrás del próximo evento en tipografía 60 pt con `minimumScaleFactor` — legible a 1 metro.
- En el instante exacto: AVSpeechSynthesizer + háptico `.notification(.success)` + scroll animado.
- Cabecera fija: fase actual del eclipse y cuenta atrás al próximo contacto.
- Pantalla siempre encendida: `UIApplication.shared.isIdleTimerDisabled = true`.
- Sesión de audio `.playback` + `.default` + `.duckOthers` para sonar sobre música ambiente.
- Botón STOP: requiere pulsación mantenida de 3 s con anillo de progreso. Soltar antes cancela.
- Al detenerse o terminar el último aviso, regreso automático al paso 1.

4. Motor de eclipse — EclipseEngine

El motor es el núcleo del proyecto. Implementa el cálculo clásico de circunstancias locales a partir de elementos besselianos en Swift puro, sin dependencias externas. Todo el cálculo ocurre en el dispositivo — sin red, sin servicios externos.

Dataset de eclipses — eclipses.json

Fichero JSON embebido en el bundle con los elementos besselianos de todos los eclipses solares de 2025 a 2035.

Fuente: NASA GSFC / Fred Espenak (Five Millennium Canon).

- id, date (ISO 8601), type (total / annular / hybrid / partial).
- t0: tiempo de referencia en horas TT desde medianoche UT.
- deltaT: diferencia TT – UT en segundos.
- Polinomios de grado 4: x(t), y(t), d(t), l1(t), l2(t), mu(t).
- tanF1, tanF2: semi-ángulos de penumbra y umbra.

Algoritmo de circunstancias locales

Implementación del método de Espenak/Meeus (Astronomical Algorithms, cap. 54). Pasos:

1. Evaluar los polinomios besselianos en t para obtener x, y, d, l1, l2, mu.
2. Calcular las coordenadas fundamentales del observador (xi, eta, zeta) desde lat/lon.
3. Iterar Newton-Raphson para encontrar los instantes de C1, C2, C3, C4 y máximo.
4. Calcular magnitud y duración de totalidad en el instante del máximo.
5. Calcular altitud y azimut solar usando ecuaciones horarias estándar.
6. Convertir a UTC y luego a hora local del dispositivo vía TimeZone del sistema.

Tests de aceptación obligatorios (Swift Testing)

- Raimat Natura, Lleida (41.68294°N, 0.47930°E) · 12/08/2026 → eclipse total.
- C1 ≈ 19:34:39 CEST · C2 ≈ 20:29:06 · MAX ≈ 20:29:20 · C3 ≈ 20:29:35. Tolerancia: ±10 s.
- Sevilla · 12/08/2026 → eclipse parcial (sin C2/C3).
- Punto en América · 12/08/2026 → sin eclipse visible (retorna nil).
- El motor no es válido sin los tres tests en verde — condición previa para cualquier UI.

```
// EclipseEngineTests.swift
@Test func raimat2026() {
    let circ = EclipseEngine.circumstances(lat: 41.68294, lon: 0.47930,
                                           date: aug12_2026)

    #expect(circ?.kind == .total)
    #expect(abs(circ!.contacts[.c2]!.timeIntervalSince(expectedC2)) < 10)
}
```

5. Arquitectura técnica

MVVM con Observation framework (@Observable)

- FlowViewModel (@Observable): máquina de estados raíz; coordenada, fecha, circunstancias, nombre de localización.
- AppSettings (@Observable): idioma, speechRate, claudeApiKey — persistidos en UserDefaults.
- Vistas SwiftUI puras sin lógica de negocio. Sin Combine — solo async/await y actors.
- ExecutionViewModel: timer de tick (0,25 s), lógica de cues, traducción previa, control de audio.
- Servicios sin estado: EclipseEngine (struct), SpeechService (class), TranslationService (struct async).

Gestión del tiempo — sin deriva

El mecanismo de avisos se basa en fechas absolutas (Date). Un único timer de tick a 0,25 s compara Date.now contra el próximo fireDate en cada ciclo. Error < 0,3 s garantizado. No se acumulan intervalos relativos — esto elimina cualquier deriva temporal.

```
// ExecutionViewModel.tick() – comparación contra Date absoluto
let now = Date.now
guard let cue = nextPendingCue, now >= cue.fireDate else { return }
speech.speak(cue.message, languageCode: voiceLanguage, rate: speechRate)
markSpoken(cue)
```

Persistencia

- SwiftData: ProgramEvent, SavedLocation, SavedProgram — un único ModelContainer en el entry point.
- UserDefaults: AppSettings (idioma, velocidad, volumen, API key) y nombre del programa activo.
- SavedProgram almacena los eventos como JSON ([ProgramEventSnapshot]) para evitar relaciones SwiftData.
- Migración automática gestionada por SwiftData con lightweight migration.

```
@Model final class ProgramEvent {
    var phase: EclipsePhase    // c1, c2, max, c3, c4
    var offsetSeconds: Int      // negativo = antes de la fase
    var message: String        // texto del aviso
    var textLanguage: String    // BCP-47: 'es-ES', 'ca-ES', 'en-GB'
    var sortIndex: Int
}

@Model final class SavedProgram {
    var name: String
    var eventsJSON: Data        // [ProgramEventSnapshot] codificado
    var createdAt: Date
    var eventCount: Int
}

@Model final class SavedLocation {
    var name: String
    var latitude, longitude: Double
    var createdAt: Date
}
```

6. Servicio de voz — SpeechService

- AVSpeechSynthesizer con voz según el idioma de ajustes: es-ES, ca-ES, en-GB.
- AVAudioSession: categoría .playback, modo .default, opción .duckOthers.
- Cola propia: si dos avisos coinciden en el tiempo, se pronuncian en orden sin solaparse.
- utterance.volume fijado a 1.0 — utterance.volume ignorada por iOS 26+ con .playback.
- Volumen: el usuario pulsa Preview en Ajustes y ajusta con los botones laterales del iPhone. El nivel de medios del sistema persiste y se aplica durante la ejecución.
- Selector de voz en Ajustes: picker por idioma con las voces instaladas (premium/enhanced/estándar). El voiceIdentifier elegido se pasa al utterance para seleccionar la voz de mayor calidad.
- cleanUpPreview() al salir de Ajustes evita que el sintetizador de preview compita con el de ejecución.
- Degradación: si la voz del idioma no está instalada, se usa la voz por defecto del sistema.
- Sesión activada al entrar en ejecución y desactivada al salir — no interfiere con otras apps.

7. Traducción automática — TranslationService

Cuando el idioma de voz difiere del idioma del texto de un aviso (campo textLanguage del evento), la app traduce ese texto antes de iniciar la ejecución.

- API: claude-haiku-4-5-20251001 (Anthropic) — rápido y económico para textos cortos.
- El usuario introduce su API key en Ajustes (SecureField, almacenado en UserDefaults).
- ExecutionViewModel.preTranslate(apiKey:) ejecuta todas las traducciones en paralelo (withTaskGroup).
- Si la traducción falla (sin red, API key inválida), se usa el texto original — la ejecución nunca falla.
- Overlay 'Translating announcements...' informa al usuario durante la espera.
- Badge dorado en el paso 4 identifica los avisos que serán traducidos.

8. Backup y restore de programas

El sistema de backup permite exportar programas entre dispositivos usando un archivo con extensión .cteprog (formato JSON). La extensión está registrada como UTI propio de la app (com.cornellana.cteprog) para que iOS enrute los archivos directamente.

Exportación (Backup)

- Ajustes → Data → Backup Programs...: hoja de selección con todos los programas guardados.
- El usuario selecciona cuáles incluir (All / None / individual). Se crea EclipsePrograms-AAAA-MM-DD.cteprog.
- El archivo se comparte vía UIActivityViewController (presentado desde el UIViewController padre para evitar el bug de pantalla gris de SwiftUI nested sheets).
- Destinos: AirDrop, iCloud Files, Mail, Mensajes, etc.

```
// BackupFile.swift – estructura del archivo
struct BackupFile: Codable {
    let version: Int           // actualmente 1
    var programs: [BackupProgram]
}
struct BackupProgram: Codable {
    var name: String
    var createdAt: Date
    var events: [ProgramEventSnapshot]
```

Importación (Restore)

- Ajustes → Data → Restore from File...: selector de archivos (fileImporter) filtrado por .cteprog.
- Archivo recibido por AirDrop o 'Abrir con...': onOpenURL en rootView detecta la extensión y parsea.
- Cold start: si la app arranca desde el archivo, el restore espera al splash (onChange de isReady).
- RestoreSheet: lista de programas del backup con checkboxes, recuento de eventos y alertas de conflicto.
- Toggle 'Overwrite existing programs': si desactivado, los programas con nombre duplicado se saltan.
- Usa .sheet(item:) para evitar la pantalla gris de isPresented con contenido opcional.

UTI registrado en Info.plist

- UTExportedTypeDeclarations: identifier=com.cornellana.cteprog, conforms=public.data, ext=cteprog.
- CFBundleDocumentTypes: LSItemContentTypes=[com.cornellana.cteprog], role=Editor, rank=Owner.
- Con el UTI registrado, tocar un .cteprog en Files o recibirlo por AirDrop abre directamente la app.

9. Localización

La app soporta tres idiomas: Español (es-ES), Català (ca-ES) e English (en-GB). Un único selector en Ajustes controla tanto la interfaz como la voz sintetizada.

- String Catalogs (Localizable.xcstrings): todas las cadenas visibles están localizadas (es-ES, ca-ES, en-GB — 160+ entradas).
- LocalizedStringKey como tipo de parámetro en vistas (EclipseCardView, HelpSheet): permite que .environment(\.locale, ...) sea respetado en tiempo de ejecución.
- String(localized:comment:) solo en contextos no-SwiftUI (viewmodels, servicios).
- Fechas y horas con FormatStyle respetando la zona horaria del dispositivo.
- .environment(\.locale, settings.locale) inyectado en rootView: aplica el idioma al árbol completo de vistas sin reiniciar la app.

10. Interfaz de usuario

Diseño sobrio orientado al uso en campo y en condiciones de baja luminosidad. Estética eclipse: fondo negro profundo, acento dorado, tipografía semántica (Dynamic Type).

- Fondo de app: #121215 (RGB 18, 18, 21). Tarjetas: #1E1E24.
- Acento dorado: #b8985a — botones principales, valores calculados, iconos de acción.
- Targets táctiles ≥ 44 pt. leading/trailing en lugar de left/right.
- Cuenta atrás del próximo evento: `font(.system(size: 60))` con `minimumScaleFactor(0.4)`.
- Target exclusivo iPhone (`TARGETED_DEVICE_FAMILY = 1`). iPad no es objetivo en v1.

Títulos contextuales en navegación

- Paso 3: el header muestra el nombre de la localización guardada si se usó una; vacío si fue GPS o manual libre.
- Paso 4: el título de la `NavigationBar` muestra el nombre del programa activo (`AppStorage 'currentProgramName'`).
- Sin nombre activo, el título queda vacío — no se muestra texto genérico.

Barra de herramientas del paso 4

- Izquierda: $\leftarrow$ Volver | papelera (botón destructivo) — alejados de los botones constructivos.
- Derecha: bookmark (programas guardados) | + (nuevo evento) — zona constructiva.
- La separación física de la papelera evita borrados accidentales.

11. Ajustes — AppSettings

language:	<code>VoiceLanguage</code> enum: es-ES / ca-ES / en-GB. Controla UI y voz simultáneamente.
speechRate:	Float mapeado al rango <code>AVSpeechUtterance.[minimum maximum]SpeechRate</code> .
voiceIdentifiers:	<code>Dict [String:String]</code> idioma $\rightarrow$ identifier de voz premium/enhanced elegida por idioma.
claudeApiKey:	String en <code>UserDefaults</code> . <code>SecureField</code> en <code>SettingsView</code> . Usado por <code>TranslationService</code> .

Sección Data en Ajustes

- Backup Programs...: abre `BackupSheet` para seleccionar y exportar programas.
- Restore from File...: abre el selector de archivos del sistema (`.cteprog`).
- Los botones usan `Label` con iconos `system (arrow.up.doc / arrow.down.doc)`.

12. Proyecto, firma y repositorio

Target:	ControlTiemposEclipse
Bundle ID:	com.cornellana.ControlTiemposEclipse
Development Team:	TJ6V4QM3GB (Francisco Cornellana Castells)
Deployment Target:	iOS 17.0
Firma:	Xcode automatic signing
Repositorio:	github.com/cornellana/control-tiempos-eclipse (público)
Lenguaje:	Swift 5.9+ — async/await, @Observable, SwiftData
Dependencias:	Ninguna externa — solo frameworks del sistema Apple

Frameworks utilizados

Framework	Uso
SwiftUI	UI completa: @Observable, NavigationStack, sheets, transitions.
SwiftData	ProgramEvent, SavedLocation, SavedProgram.
MapKit	Mapa en paso 3 (verificación) y paso 5 (ejecución de fondo).
CoreLocation	Lectura puntual GPS con permiso When In Use.
AVFoundation	AVSpeechSynthesizer y AVAudioSession para la voz.
UniformTypePld.	UTType para registro de .cteprog y fileImporter.
UIKit (mínimo)	isIdleTimerDisabled, UIWindow.appearance(), UIActivityViewController.
Foundation	URLSession (TranslationService), JSONEncoder/Decoder, FileManager.

Ficheros relevantes

Fichero	Responsabilidad
EclipseEngine.swift	Motor besseliano — cálculo offline de circunstancias locales.
eclipses.json	Dataset 2025–2035 de elementos besselianos embebido en bundle.
FlowViewModel.swift	Máquina de estados + evaluación del eclipse + locationName.
ExecutionViewModel.swift	Lógica de ejecución, tick timer (0,25 s), traducción previa.
EclipseCardView.swift	Tarjeta de resultados: contactos, magnitud, cobertura, azimut. Usa LocalizedStringKey para
HelpSheet.swift	Glosario de conceptos: C1–C4, magnitud, cobertura, tipos de eclipse. LocalizedStringKey
BackupFile.swift	BackupFile, BackupProgram, BackupService, IdentifiableBackup.
BackupSheet.swift	UI de selección + presentación de UIActivityViewController.
RestoreSheet.swift	UI de importación con toggle de sobreescritura por conflicto.
Step1LocationView.swift	Paso 1: GPS, manual, localizaciones guardadas con nombre.
Step3VerificationView.swift	Paso 3: mapa + tarjeta eclipse o modo simulación.
Step4ProgramView.swift	Paso 4: lista de eventos, programas guardados, CRUD.
Step5ExecutionView.swift	Paso 5: ejecución con mapa, cue list, STOP protegido.
SpeechService.swift	Cola AVSpeechSynthesizer + gestión de AVAudioSession.
TranslationService.swift	Traducción vía API Claude (haiku-4-5-20251001).
AppSettings.swift	Ajustes @Observable persistidos en UserDefaults.
Localizable.xcstrings	String Catalog: 160+ cadenas en es-ES, ca-ES, en-GB.
ControlTiemposEclipse-Info.plist	Registro UTI com.cornellana.cteprog.
EclipseEngineTests.swift	Tests de aceptación: Raimat, Sevilla, América.

13. Plan de hitos

Deadline inamovible: eclipse total del 12 de agosto de 2026.

Hito 1 — Motor

COMPLETADO

- EclipseEngine: elementos besselianos + iteración Newton-Raphson.
- Dataset 2025–2035 embebido (eclipses.json).
- Tests de aceptación en verde: Raimat (total), Sevilla (parcial), América (sin eclipse).

Hito 2 — Flujo completo

COMPLETADO

- Pasos 1–4 funcionales con localización GPS y manual, mapa, eclipse card.
- Modo simulación completo con entrada de C2 y fases derivadas.
- Programa de eventos: CRUD, offsets relativos a fases, tiempos resueltos.
- Localizaciones y programas guardados con nombre (SwiftData). Backup/restore.
- Traducción automática de avisos (TranslationService + API Claude).
- Ajustes: idioma, velocidad, volumen, API key. Fondo negro en arranque.

Hito 3 — Ejecución

COMPLETADO

- Pantalla paso 5: mapa de fondo, lista de cues, cuenta atrás 60 pt.
- AVSpeechSynthesizer + háptico en el instante exacto, scroll animado.
- Botón STOP con hold de 3 s y anillo de progreso.
- Prueba de simulación completa de principio a fin.

Hito 4 — Pulido y ensayo

EN CURSO

- [OK] Icono: script CoreGraphics (anillo de diamantes, #b8985a, 1024x1024 sin alfa).
- [OK] Localización completa ca-ES y es-ES (Localizable.xcstrings: 3 idiomas, 160+ cadenas).
- [OK] Fix LocalizedStringKey en EclipseCardView y HelpSheet para idioma en app.
- Prueba en iPhone real de Francisco.
- Ensayo general con cámaras — primera semana de agosto de 2026.
- Congelación de código el 9 de agosto de 2026.

14. Restricciones de diseño (no negociables)

- Sin red para el cálculo: EclipseEngine funciona completamente offline con dataset embebido.
- Sin notificaciones locales como mecanismo de avisos — AVSpeechSynthesizer en foreground.
- Sin dependencias externas (SPM/CocoaPods) sin aprobación explícita.
- Sin timers relativos acumulados: siempre Date absolutos — elimina cualquier deriva.
- No avanzar hitos sin confirmación: un paso cada vez.
- Tests de aceptación obligatorios — el motor no es válido sin ellos en verde.
- Cero errores de compilación antes de entregar cualquier cambio.
- Push a GitHub tras confirmación de Francisco.

15. Precisión del sistema de avisos

El sistema garantiza un error de pronunciación $< 0,3$ s respecto al instante calculado. Decisiones de diseño que lo hacen posible:

- Fechas absolutas: los instantes se calculan una sola vez, nunca se acumulan intervalos.
- preTranslate() completa TODAS las traducciones antes de llamar a start() — sin esperas en ejecución.
- AVAudioSession activado antes del primer aviso — sin latencia de inicialización durante ejecución.
- Pantalla siempre encendida: isIdleTimerDisabled evita que el sistema suspenda la app.
- postUtteranceDelay = 0,1 s en cada utterance para separar avisos consecutivos sin solapamiento.